Book & Author Management System - Project Documentation

Table of Contents

1. Project Overview

This project is a **Spring Boot 3.2 web application** that manages two related entities: **Books** and **Authors**. It demonstrates full CRUD (Create, Read, Update) operations with a proper multi-layered architecture including Entity, Repository, Service, and Controller layers.

Tech Stack

COMPONENT	TECHNOLOGY
Language	Java 17

COMPONENT	TECHNOLOGY
Framework	Spring Boot 3.2
ORM	Spring Data JPA (Hibernate)
Web	Spring MVC
Views	JSP + JSTL
Database	H2 In-Memory Database
Testing	JUnit 5 + Mockito
Build Tool	Maven

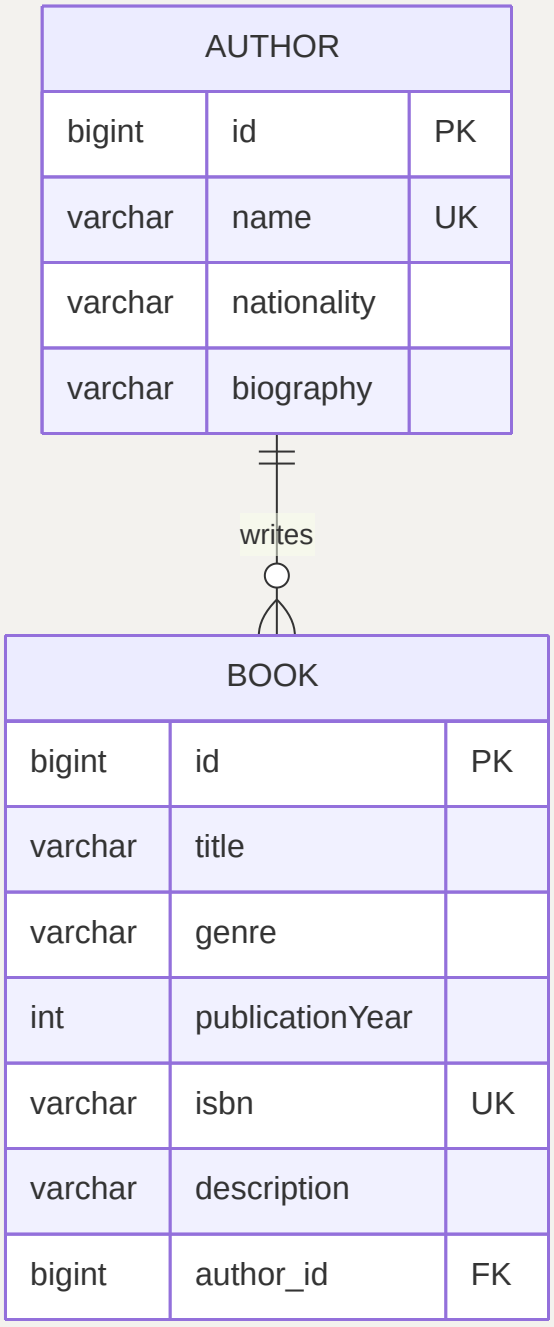
Application Features

- **Create**: Add new books and authors via JSP forms with validation
- **Read**: Display all books, all authors, and a combined INNER JOIN view
- **Update**: Edit existing book and author details
- **Exception Handling**: Global exception handling for data integrity violations
- **Custom Query**: JPQL INNER JOIN query returning typed DTO
- **Database Population**: 10 sample authors and 10 sample books auto-loaded on startup

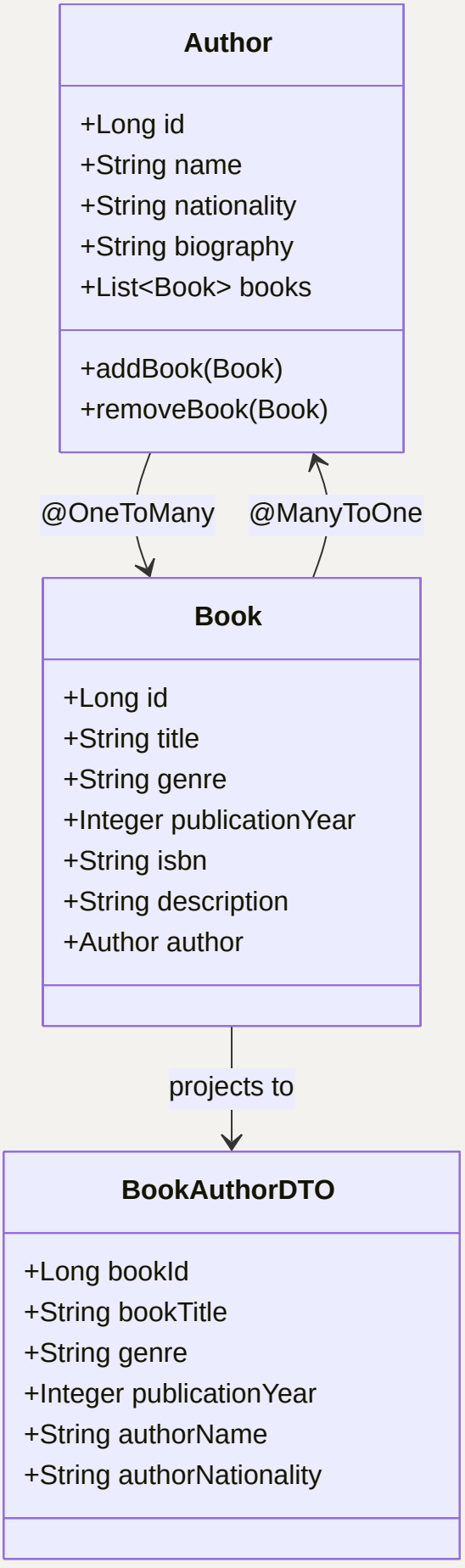
---

2. Entity Relationship Design

2.1 Entity Relationship Diagram


AUTHOR
bigint
id
PK
varchar
name
UK
varchar
nationality
varchar
biography
writes
BOOK
bigint
id
PK
varchar
title
varchar
genre
int
publicationYear
varchar
isbn
UK
varchar
description
bigint
author_id
FK

2.2 Class Diagram



2.3 Entity Classes

Author Entity (`src/main/java/com/example/bookauthor/entity/Author.java`)

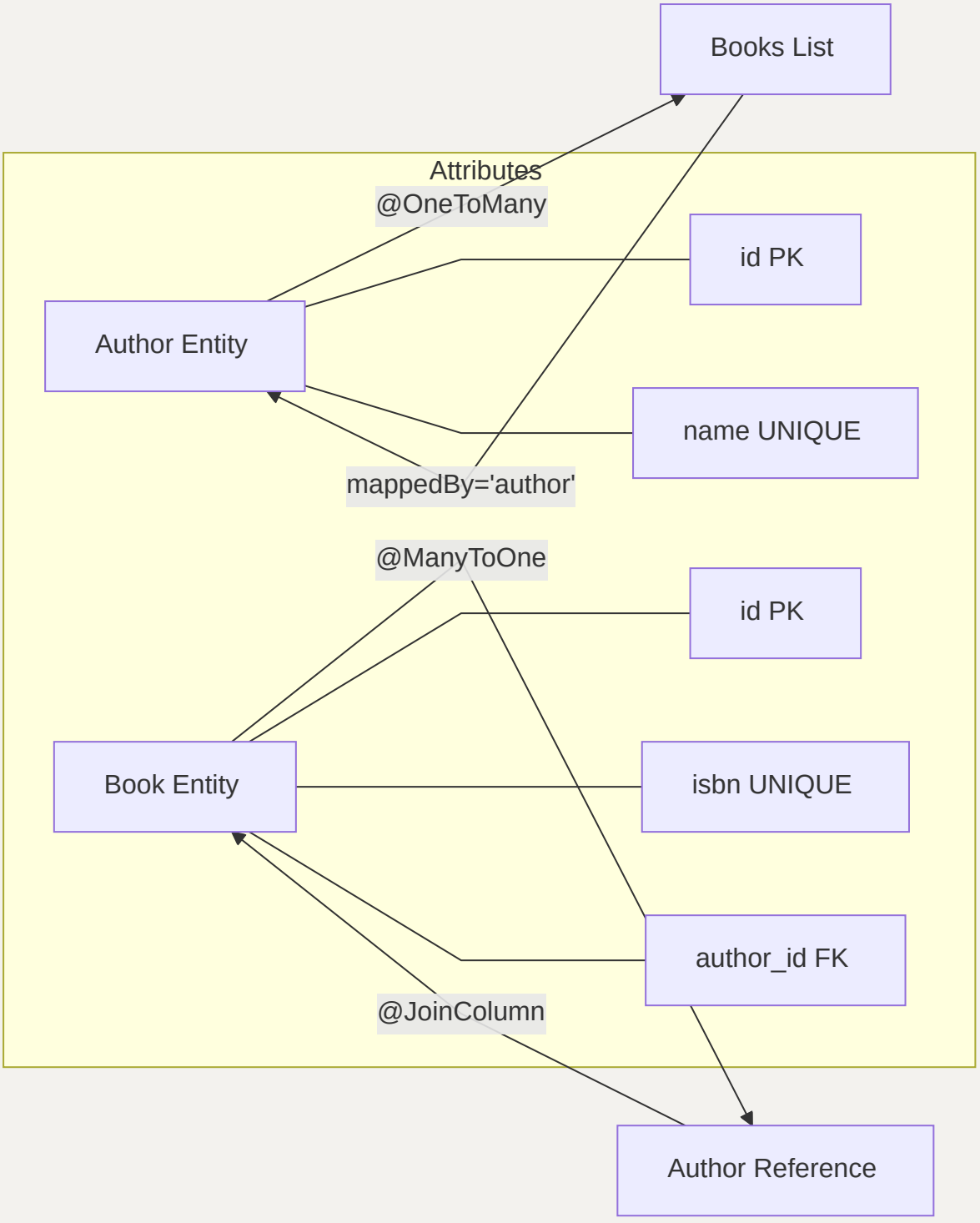
```java
@Entity
@Table(name = "authors")
public class Author {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 100)
    @Column(nullable = false, unique = true)
    private String name;

    @Size(max = 100)
    private String nationality;

    @Size(max = 500)
    @Column(length = 500)
    private String biography;

    @OneToMany(mappedBy = "author", cascade = CascadeType.ALL,
               orphanRemoval = true, fetch = FetchType.LAZY)
    private List<Book> books = new ArrayList<>();

    // Helper methods to maintain bidirectional consistency
    public void addBook(Book book) {
        books.add(book);
        book.setAuthor(this);
    }

    public void removeBook(Book book) {
        books.remove(book);
        book.setAuthor(null);
    }
```

```
    // Getters and setters, equals, hashCode, toString
}
```

Book Entity (`src/main/java/com/example/bookauthor/entity/Book.java`)

```
@Entity
@Table(name = "books")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Title is required")
    @Size(min = 1, max = 200)
    @Column(nullable = false, length = 200)
    private String title;

    @Size(max = 50)
    private String genre;

    @NotNull(message = "Publication year is required")
    @Positive(message = "Publication year must be positive")
    private Integer publicationYear;

    @NotBlank(message = "ISBN is required")
    @Size(max = 20)
    @Column(nullable = false, unique = true)
    private String isbn;

    @Column(length = 1000)
    private String description;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "author_id", nullable = false)
    @NotNull(message = "Author is required")
    private Author author;

    // Getters and setters
}
```

2.4 Relationship Explanation



Key Relationship Features:

- **One-to-Many**: One Author can write multiple Books
- **Many-to-One**: Each Book belongs to one Author
- **CascadeType.ALL**: Operations on Author cascade to Books
- **orphanRemoval = true**: Removing a book from the list deletes it from DB

- **FetchType.LAZY**: Related entities are loaded on-demand for performance

---

3. Implementation Details

3.1 Populate Database

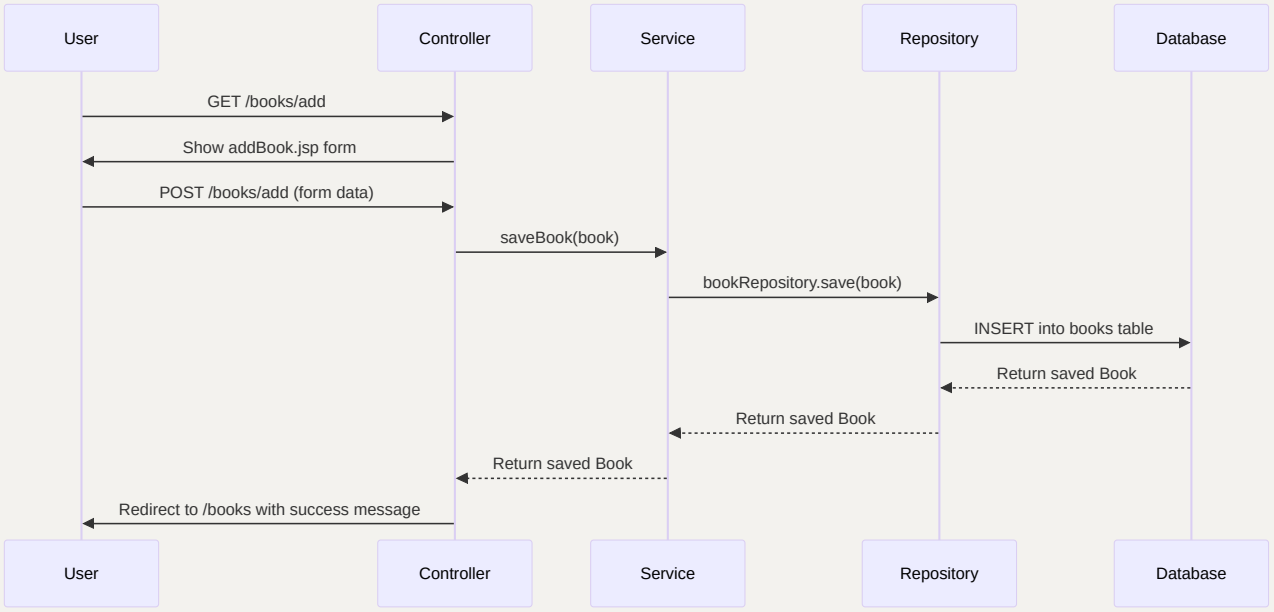
The database is populated using `data.sql` at `src/main/resources/data.sql`. Spring Boot automatically executes this on startup.

```
-- Insert 10 Authors
INSERT INTO authors (name, nationality, biography) VALUES
('J.K. Rowling', 'British', 'Author of the Harry Potter
series...'),
('Stephen King', 'American', 'Master of horror fiction...'),
('Agatha Christie', 'British', 'Best-selling mystery novelist...'),
('George Orwell', 'British', 'Known for dystopian fiction...'),
('Jane Austen', 'British', 'Romantic fiction novelist...'),
('Ernest Hemingway', 'American', 'Nobel Prize-winning author...'),
('F. Scott Fitzgerald', 'American', 'Jazz Age novelist...'),
('Gabriel Garcia Marquez', 'Colombian', 'Nobel Prize-winning...'),
('Haruki Murakami', 'Japanese', 'Contemporary fiction writer...'),
('Toni Morrison', 'American', 'Nobel Prize-winning novelist...');

-- Insert 10 Books linked to authors
INSERT INTO books (title, genre, publication_year, isbn,
description, author_id) VALUES
('Harry Potter and the Sorcerer''s Stone', 'Fantasy', 1997, '978-
0747532699', '...', 1),
('The Shining', 'Horror', 1977, '978-0307743657', '...', 2),
('Murder on the Orient Express', 'Mystery', 1934, '978-0062693662',
'...', 3),
('1984', 'Dystopian', 1949, '978-0451524935', '...', 4),
('Pride and Prejudice', 'Romance', 1813, '978-0141439518', '...',
5),
('The Old Man and the Sea', 'Fiction', 1952, '978-0684801223',
'...', 6),
('The Great Gatsby', 'Fiction', 1925, '978-0743273565', '...', 7),
```

```
('One Hundred Years of Solitude', 'Magical Realism', 1967, '978-
0060883287', '...', 8),
('Norwegian Wood', 'Fiction', 1987, '978-0099448822', '...', 9),
('Beloved', 'Historical Fiction', 1987, '978-1400033416', '...',
10);
```

3.2 Create Operation

Sequence Diagram



Controller Method (`BookAuthorController.java`)

```
@PostMapping("/books/add")
public String addBook(@Valid @ModelAttribute("book") Book book,
                      BindingResult result,
                      Model model,
                      RedirectAttributes redirectAttributes) {
    if (result.hasErrors()) {
        model.addAttribute("authors",
authorService.getAllAuthors());
        return "addBook";
    }
    bookService.saveBook(book);
    redirectAttributes.addFlashAttribute("successMessage", "Book
added successfully!");
    return "redirect:/books";
}
```

Service Layer - Duplicate Detection (`BookServiceImpl.java`)

```
@Override
public Book saveBook(Book book) {
    if (book.getIsbn() != null &&
bookRepository.existsByIsbn(book.getIsbn())) {
        throw new DataIntegrityViolationException(
            "Book with ISBN '" + book.getIsbn() + "' already
exists");
    }
    return bookRepository.save(book);
}
```

JSP Form (`addBook.jsp`)

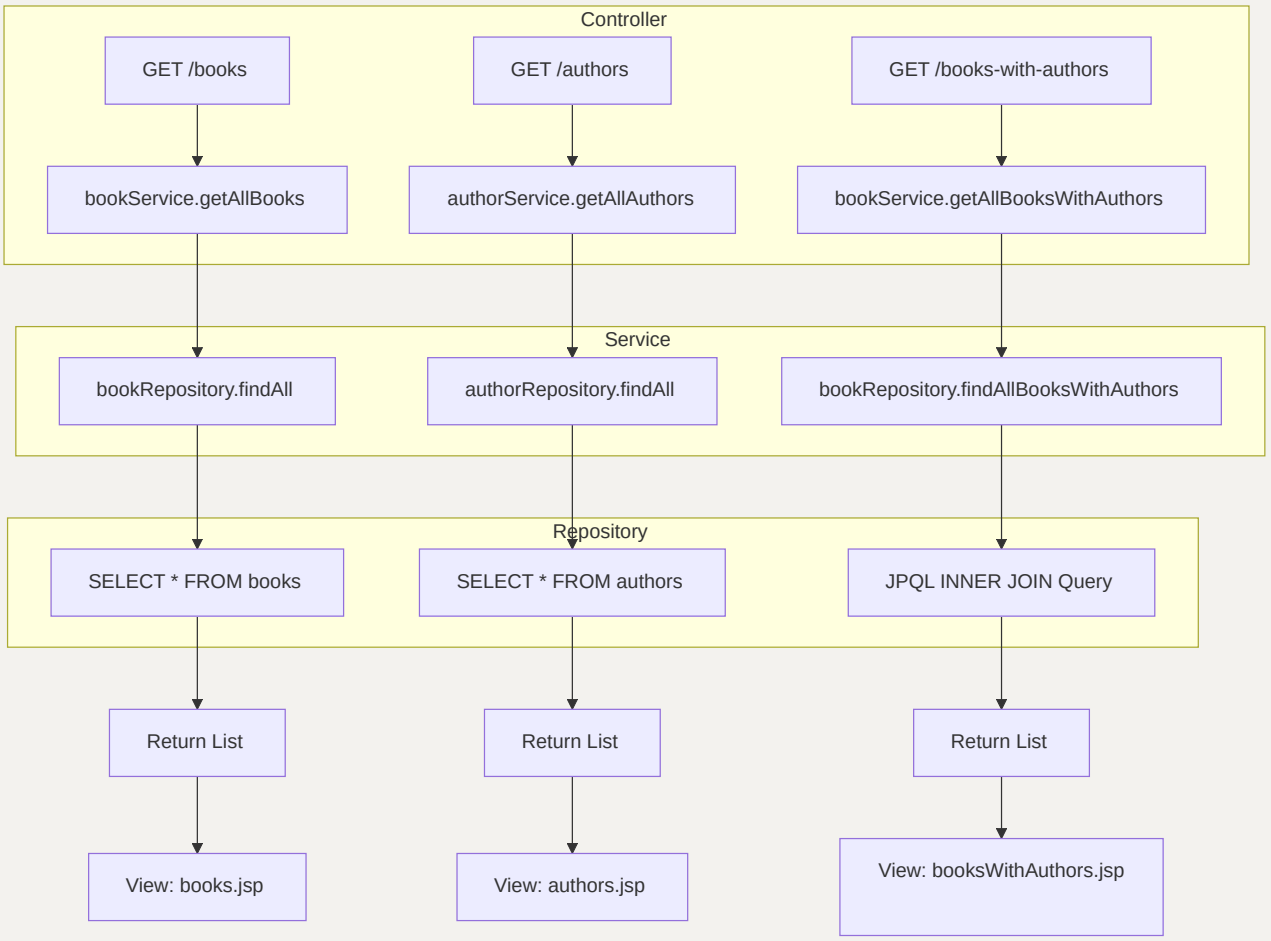
```
<form action="/books/add" method="post" class="form">
    <div class="form-group">
        <label for="title">Title *</label>
        <input type="text" id="title" name="title" required>
    </div>
```

```
    <div class="form-group">
        <label for="genre">Genre</label>
        <input type="text" id="genre" name="genre">
    </div>
    <div class="form-group">
        <label for="publicationYear">Publication Year *</label>
        <input type="number" id="publicationYear"
name="publicationYear" required>
    </div>
    <div class="form-group">
        <label for="isbn">ISBN * (Unique)</label>
        <input type="text" id="isbn" name="isbn" required>
    </div>
    <div class="form-group">
        <label for="description">Description</label>
        <textarea id="description" name="description" rows="4">
</textarea>
    </div>
    <div class="form-group">
        <label for="author">Author *</label>
        <select id="author" name="author.id" required>
            <option value="">-- Select Author --</option>
            <c:forEach var="author" items="${authors}">
                <option value="${author.id}">${author.name}
</option>
            </c:forEach>
        </select>
    </div>
    <div class="form-actions">
        <button type="submit" class="btn btn-primary">Add
Book</button>
        <a href="/books" class="btn btn-secondary">Cancel</a>
    </div>
</form>
```

Exception Handling: Duplicate ISBN detection using `existsByIsbn()` method in repository.

3.3 Read Operation

Flow Diagram



Custom INNER JOIN Query (`BookRepository.java`)

```
public interface BookRepository extends JpaRepository<Book, Long> {

    @Query("SELECT new com.example.bookauthor.dto.BookAuthorDTO(" +
           "b.id, b.title, b.genre, b.publicationYear, a.name,
a.nationality) " +
           "FROM Book b INNER JOIN b.author a")
    List<BookAuthorDTO> findAllBooksWithAuthors();

    boolean existsByIsbn(String isbn);
}
```

DTO for JOIN Result (`BookAuthorDTO.java`)

```
public class BookAuthorDTO {
    private Long bookId;
    private String bookTitle;
    private String genre;
    private Integer publicationYear;
    private String authorName;
    private String authorNationality;

    public BookAuthorDTO(Long bookId, String bookTitle, String
genre,
                         Integer publicationYear, String
authorName,
                         String authorNationality) {
        this.bookId = bookId;
        this.bookTitle = bookTitle;
        this.genre = genre;
        this.publicationYear = publicationYear;
        this.authorName = authorName;
        this.authorNationality = authorNationality;
    }
    // Getters and setters
}
```

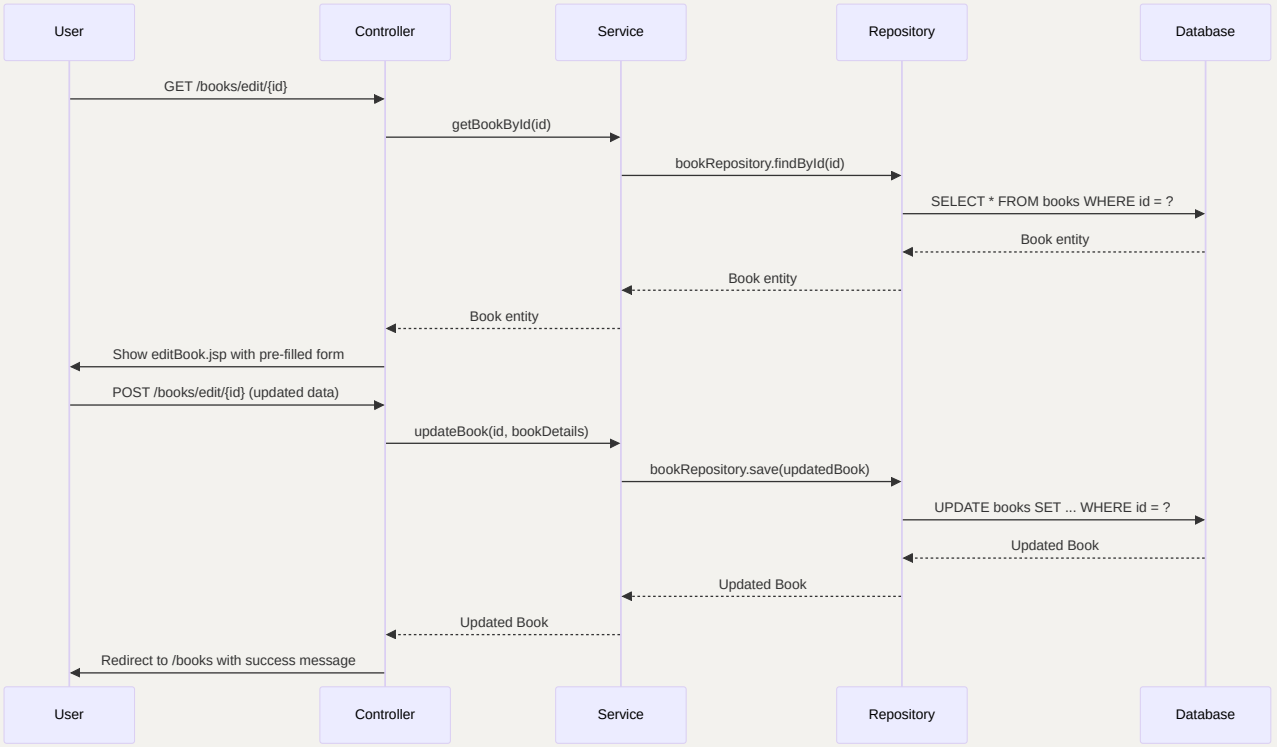
JSP View for JOIN (`booksWithAuthors.jsp`)

```
<table class="data-table">
    <thead>
        <tr>
            <th>Book Title</th>
            <th>Genre</th>
            <th>Year</th>
            <th>Author</th>
            <th>Nationality</th>
        </tr>
    </thead>
    <tbody>
        <c:forEach var="item" items="${bookAuthorList}">
```

```
            <tr>
                <td>${item.bookTitle}</td>
                <td><span class="badge">${item.genre}</span></td>
                <td>${item.publicationYear}</td>
                <td>${item.authorName}</td>
                <td>${item.authorNationality}</td>
            </tr>
        </c:forEach>
    </tbody>
</table>
```

3.4 Update Operation

Sequence Diagram



Controller Update Method

```
@PostMapping("/books/edit/{id}")
public String updateBook(@PathVariable Long id,
                         @Valid @ModelAttribute("book") Book book,
                         BindingResult result,
                         Model model,
                         RedirectAttributes redirectAttributes) {
    if (result.hasErrors()) {
        model.addAttribute("authors",
authorService.getAllAuthors());
        return "editBook";
    }
    bookService.updateBook(id, book);
    redirectAttributes.addFlashAttribute("successMessage", "Book
updated successfully!");
    return "redirect:/books";
}
```

Service Update Implementation (`BookServiceImpl.java`)

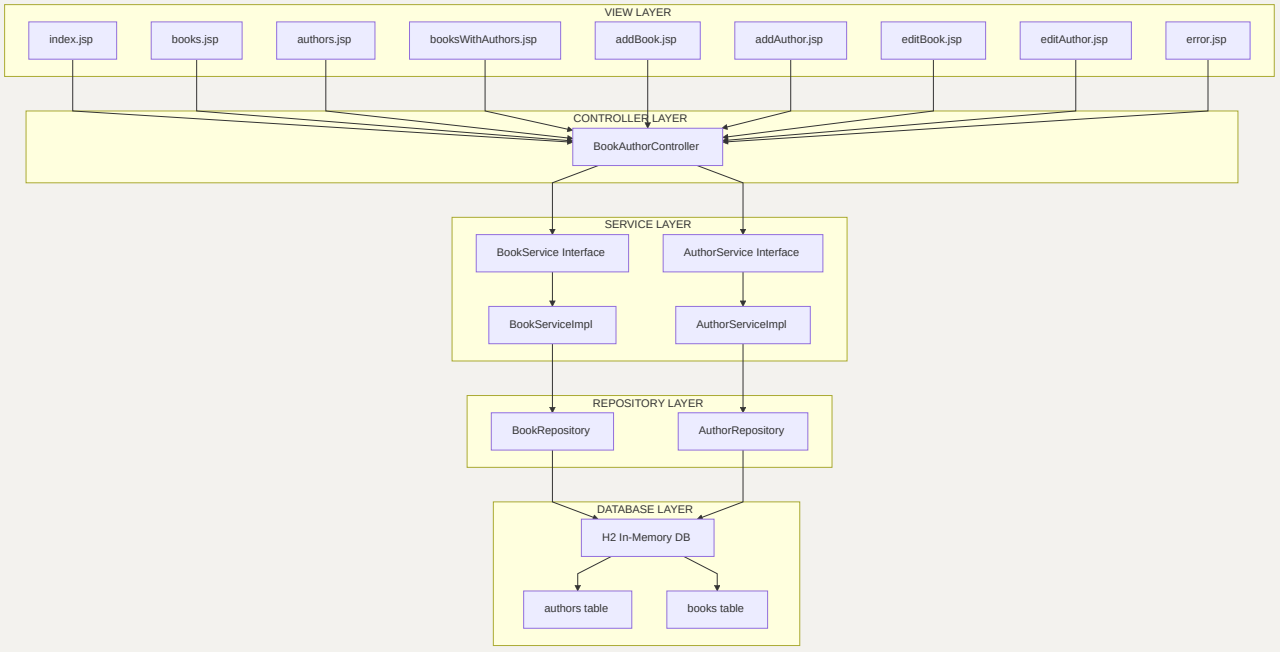
```
@Override
public Book updateBook(Long id, Book bookDetails) {
    Book book = getBookById(id);

    if (bookDetails.getIsbn() != null &&
!bookDetails.getIsbn().equals(book.getIsbn())) {
        if (bookRepository.existsByIsbn(bookDetails.getIsbn())) {
            throw new DataIntegrityViolationException("Book with
ISBN already exists");
        }
    }

    book.setTitle(bookDetails.getTitle());
    book.setGenre(bookDetails.getGenre());
    book.setPublicationYear(bookDetails.getPublicationYear());
    book.setIsbn(bookDetails.getIsbn());
    book.setDescription(bookDetails.getDescription());
    book.setAuthor(bookDetails.getAuthor());
```

```
    return bookRepository.save(book);
}
```

---

4. Spring Boot Components Integration

Architecture Diagram



Layer Responsibilities

LAYER	RESPONSIBILITY	ANNOTATIONS
View	Display data, collect user input	JSP, JSTL, EL
Controller	Handle HTTP requests, coordinate views	`@Controller`, `@GetMapping`, `@PostMapping`
Service	Business logic, validation	`@Service`, `@Transactional`
Repository	Database operations	`@Repository`, extends `JpaRepository`
Entity	Data model, ORM mapping	`@Entity`, `@Id`, `@OneToMany`, `@ManyToOne`

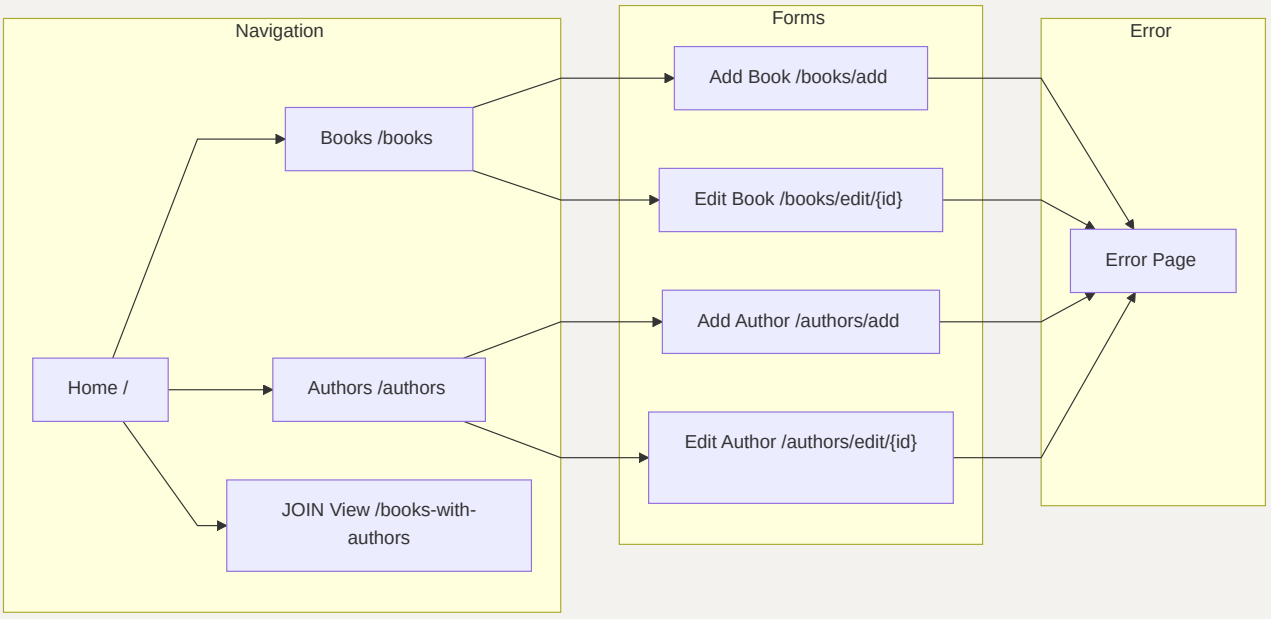
Key Integration Points

```
// Controller → Service: Constructor Injection
public BookAuthorController(BookService bookService, AuthorService
authorService) {
    this.bookService = bookService;
    this.authorService = authorService;
}

// Service → Repository: Constructor Injection
public BookServiceImpl(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
}

// Data Binding: @ModelAttribute + @Valid + BindingResult
@PostMapping("/books/add")
public String addBook(@Valid @ModelAttribute("book") Book book,
                      BindingResult result, Model model) { ... }
```

---

5. User Interface

UI Pages Overview



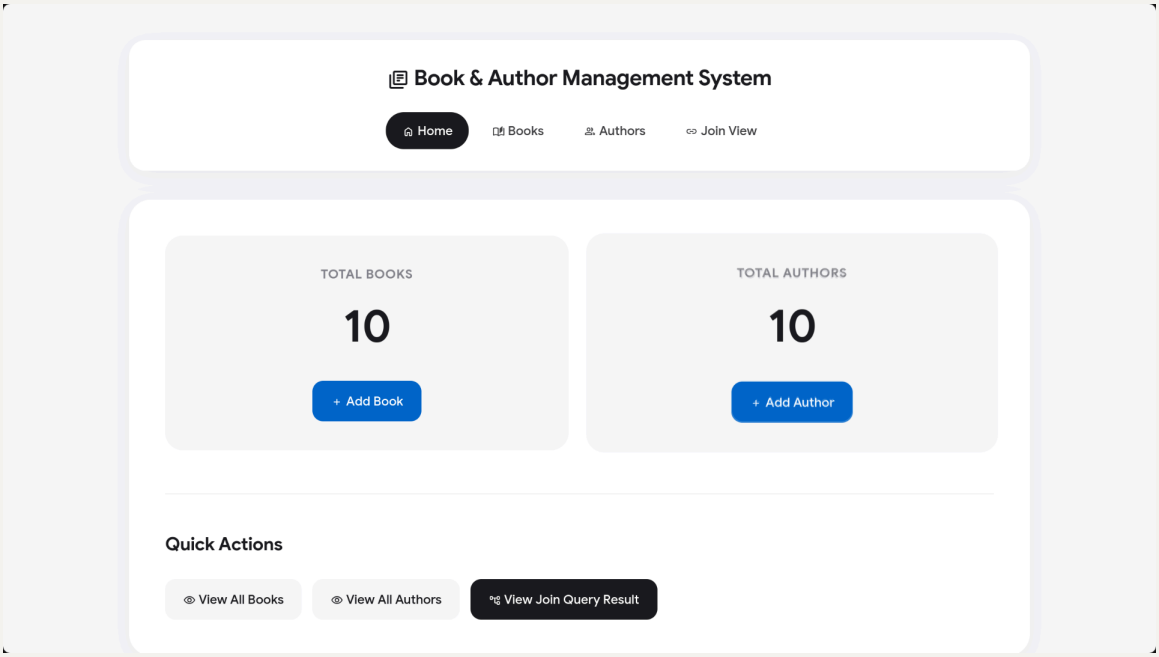
Styling Features

The application uses a dedicated CSS file (`src/main/resources/static/css/style.css`) with:

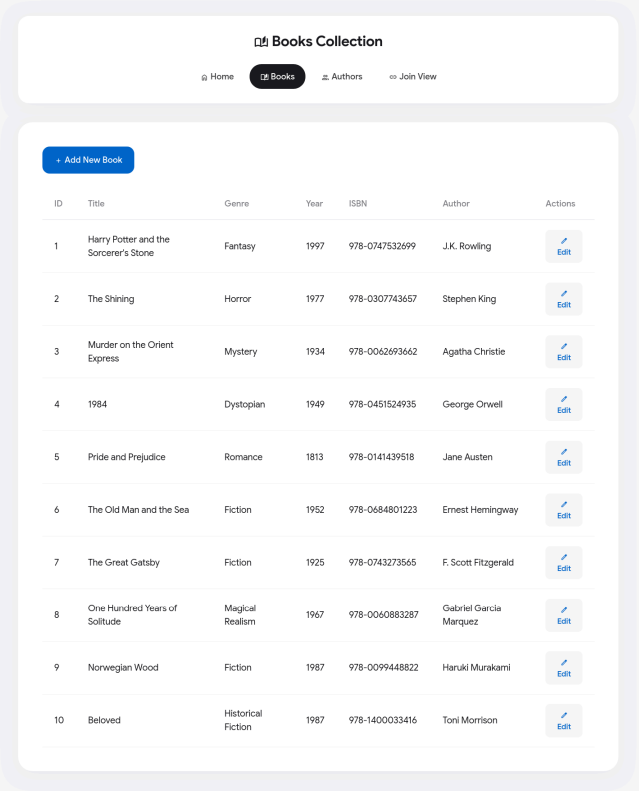
- **Modern gradient background**: `linear-gradient(135deg, #667eea 0%, #764ba2 100%)`
- **Responsive card-based layout**
- **Styled tables with hover effects**
- **Form styling with focus states**
- **Alert messages (success/error)**
- **Badge components for genres**
- **Responsive design with media queries**

Screenshots

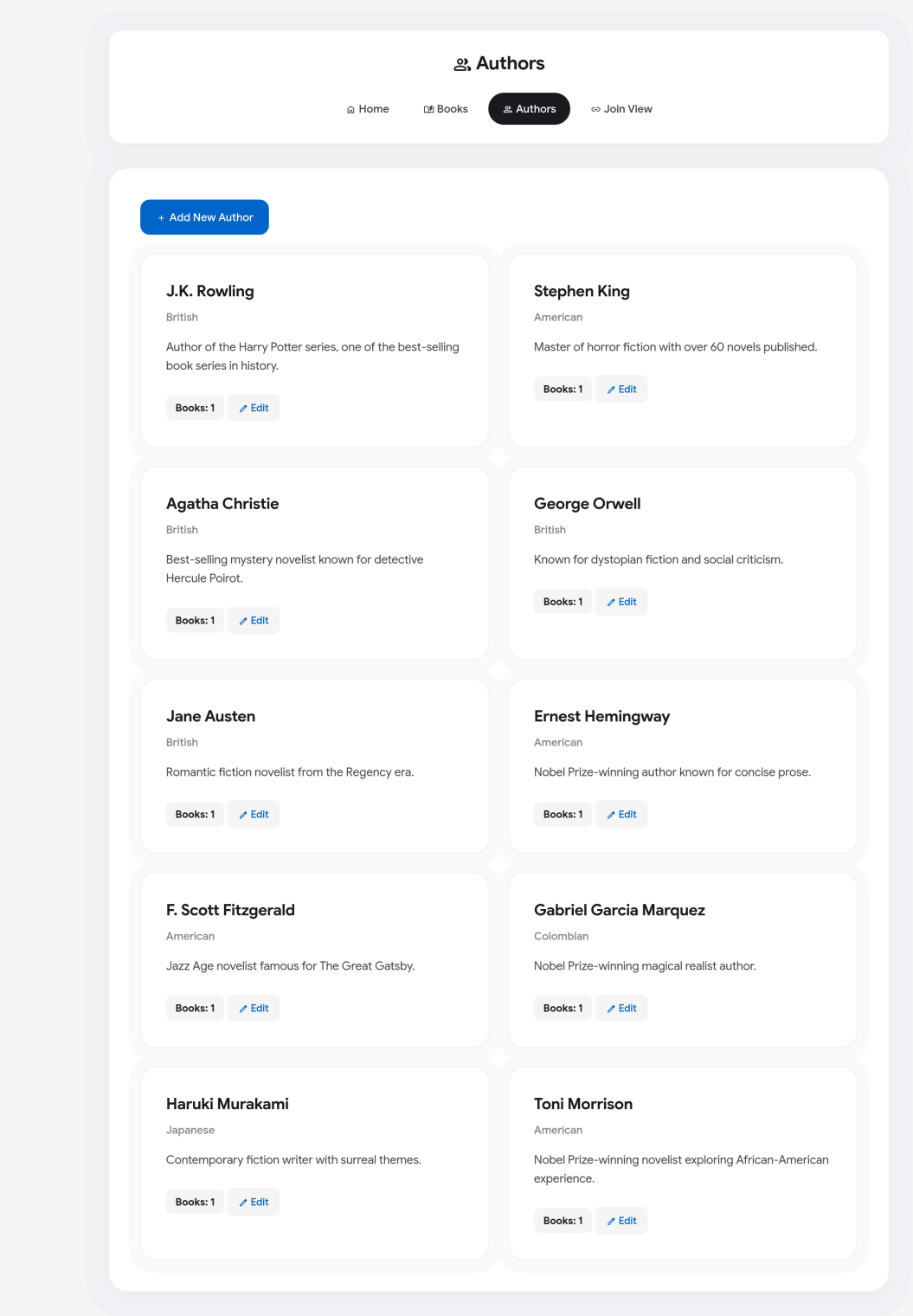
1. **Home Page** (`http://localhost:8080/`) - Dashboard showing book and author counts



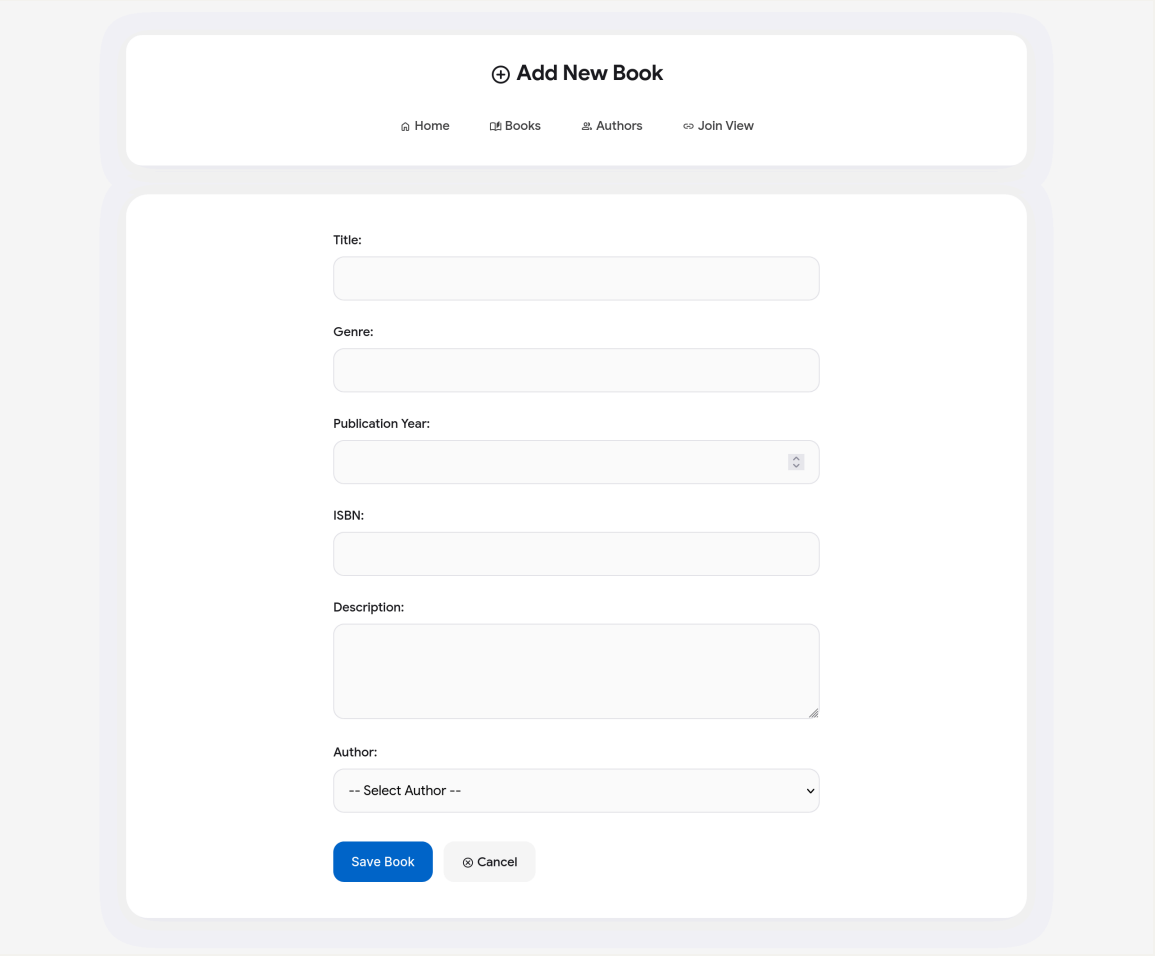
2. **Books List** (`http://localhost:8080/books`) - Table displaying all 10 books with edit button



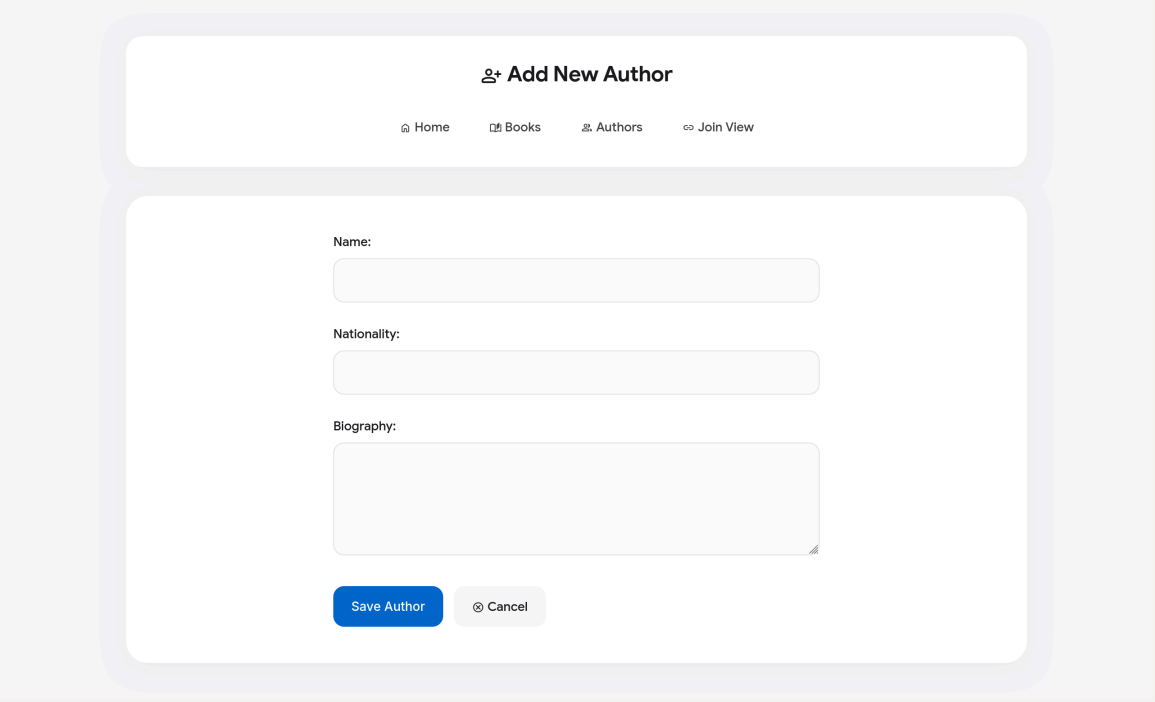
3. **Authors List** (`http://localhost:8080/authors`) - Card-based layout showing all 10 authors



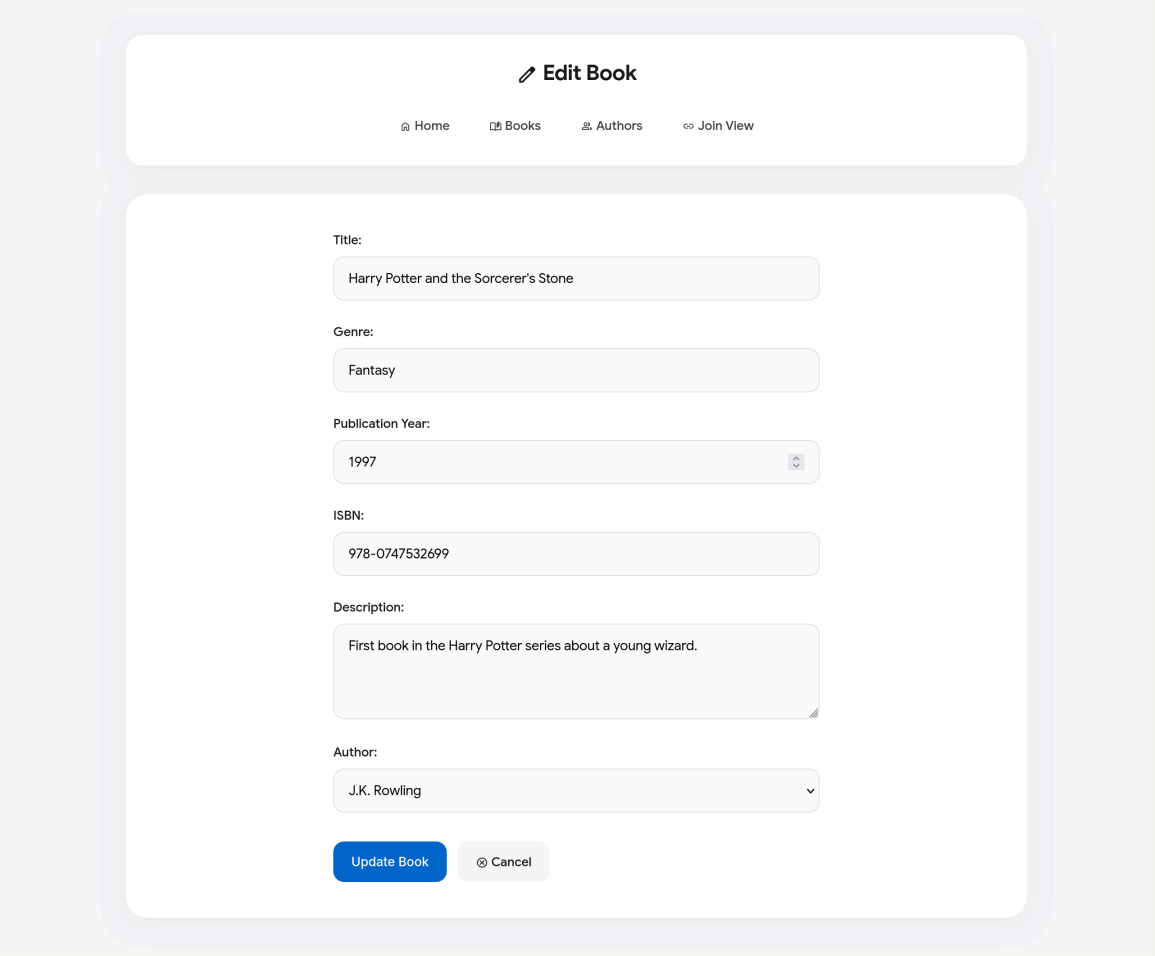
4. **Add Book Form** (`http://localhost:8080/books/add`) - Empty form



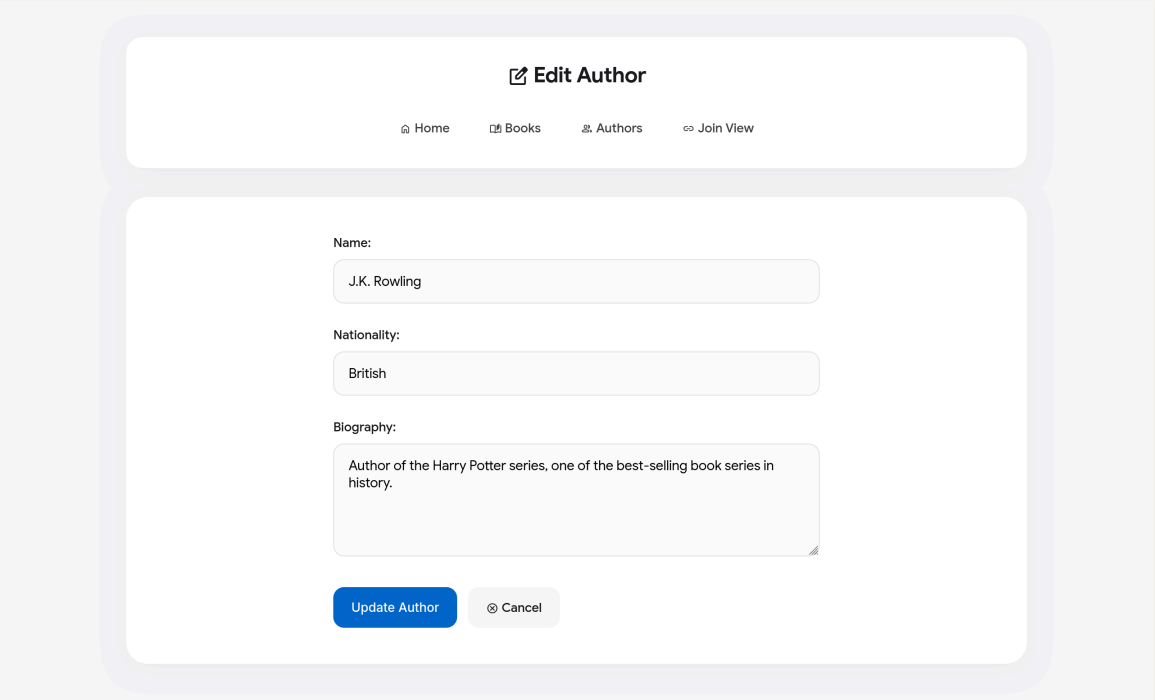
5. **Add Author Form** (`http://localhost:8080/authors/add`) - Empty form



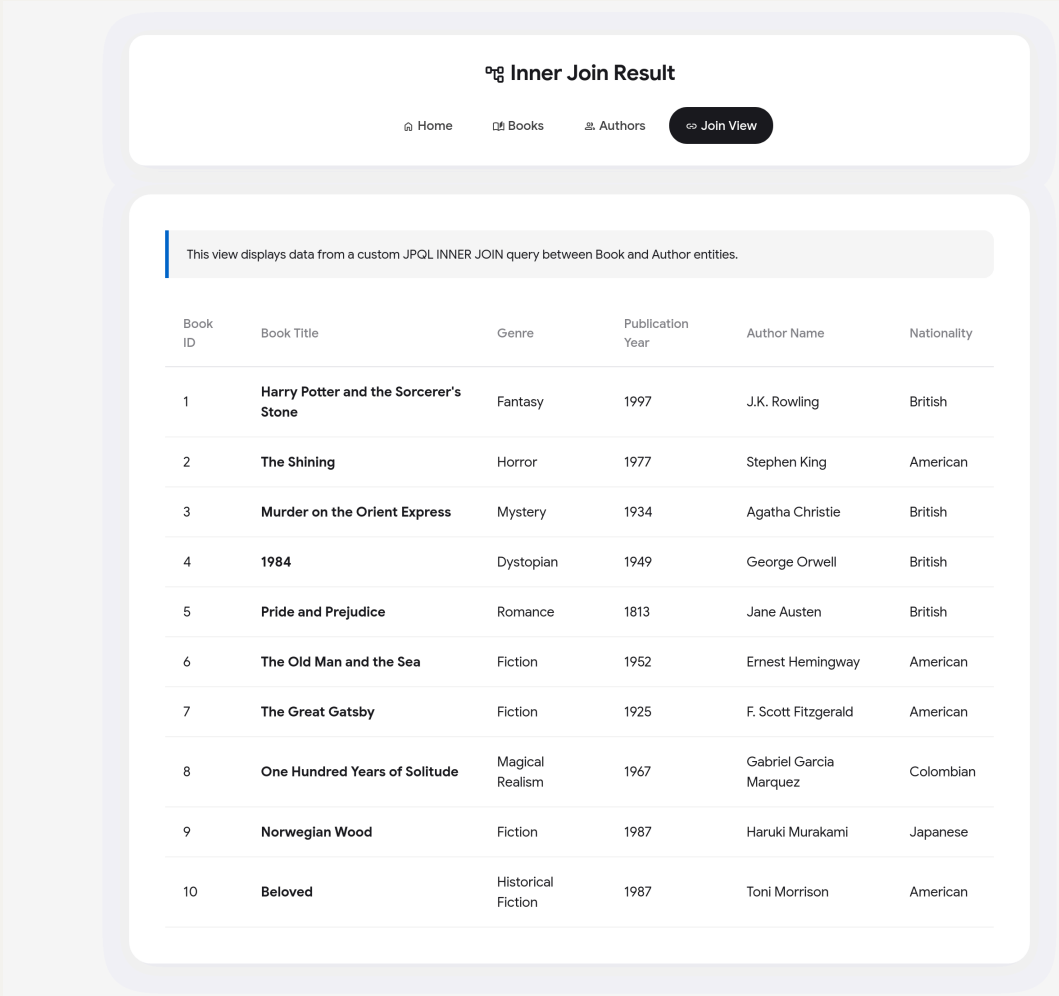
6. **Edit Book Form** (`http://localhost:8080/books/edit/1`) - Pre-populated form



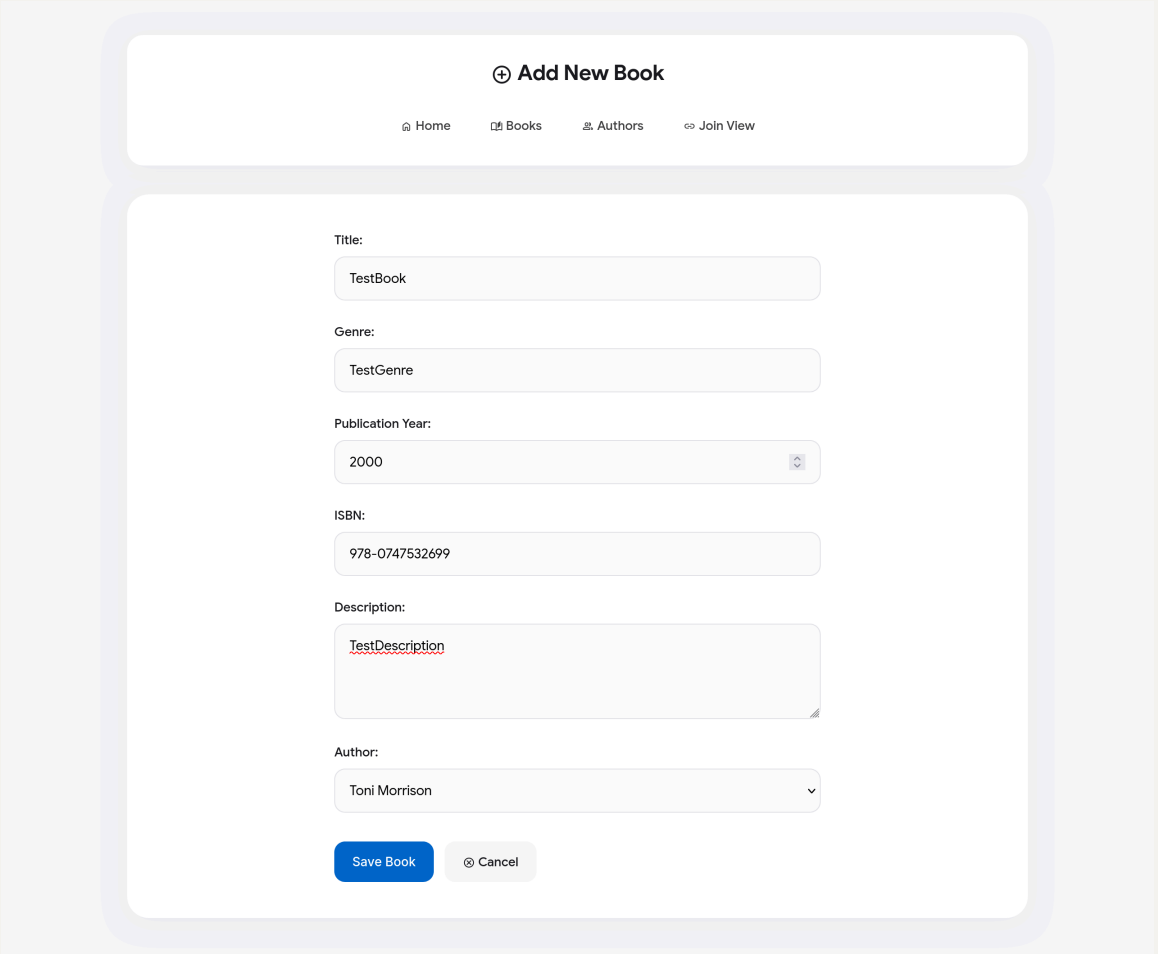
7. **Edit Author Form** (`http://localhost:8080/authors/edit/1`) - Pre-populated form

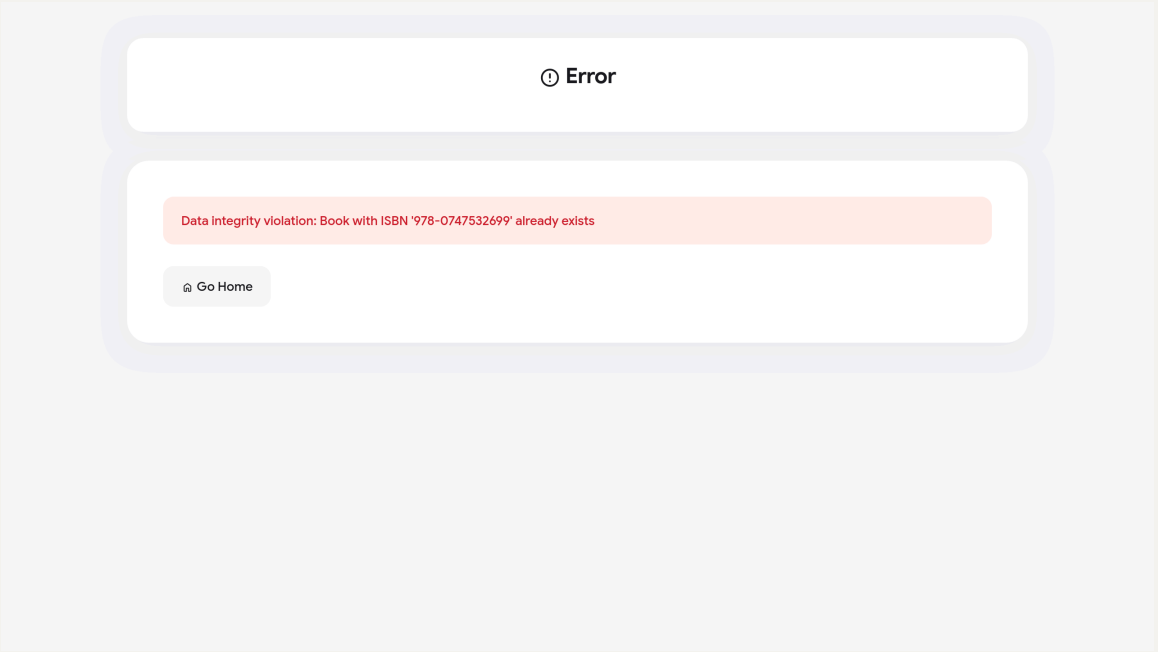


8. **Books with Authors (JOIN)** (`http://localhost:8080/books-with-authors`) - Combined table

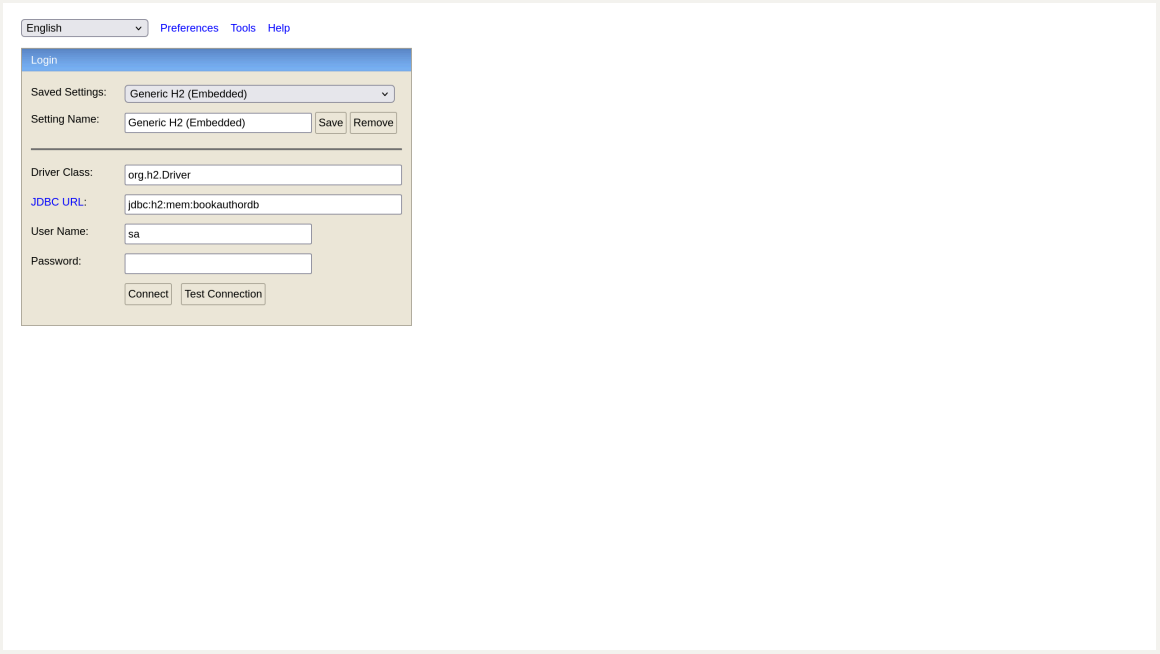


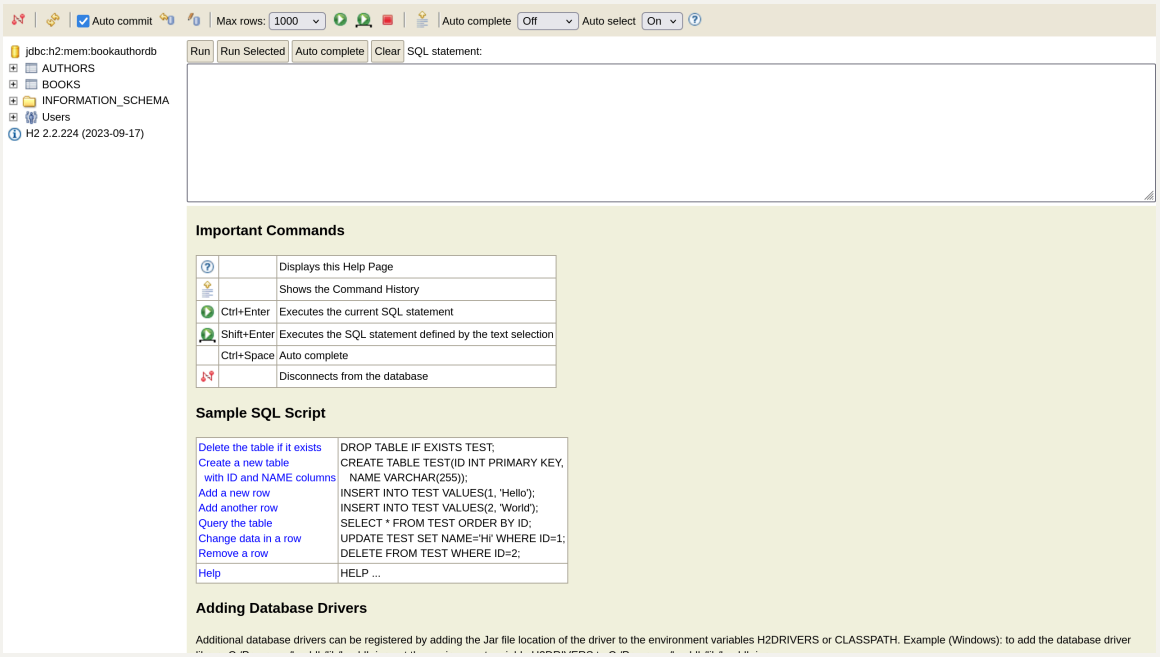
9. **Error Page** - Trigger by adding duplicate ISBN

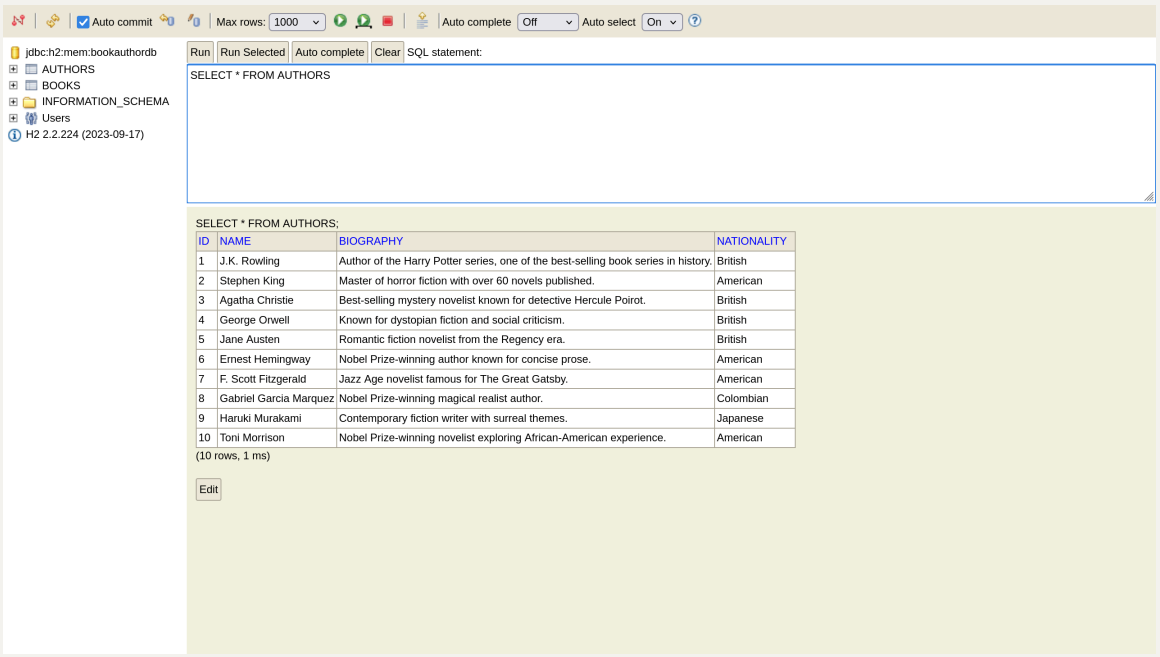


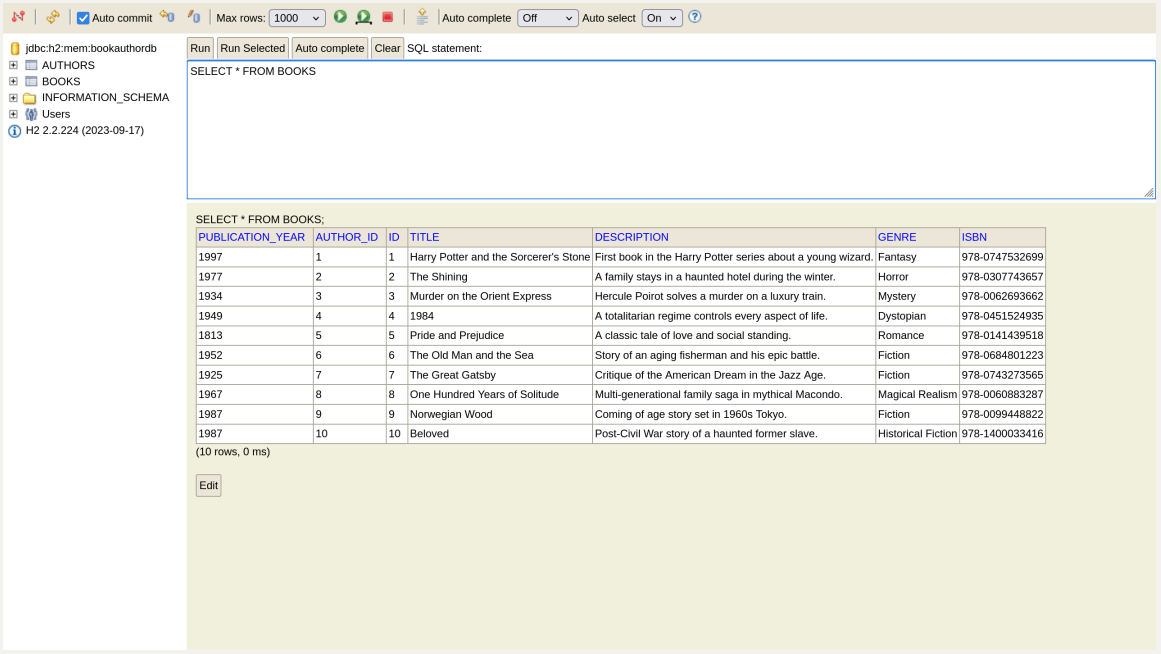


10. **H2 Console** (`http://localhost:8080/h2-console`) - Shows database tables


English
Preferences
Tools
Help
Login
Saved Settings:
Generic H2 (Embedded)
Setting Name:
Generic H2 (Embedded)
Save
Remove
Driver Class:
org.h2.Driver
JDBC URL:
jdbc:h2:mem:bookauthordb
User Name:
sa
Password:
Connect
Test Connection


Auto commit
Max rows: 1000
Auto complete Off
Auto select On
jdbc:h2:mem:bookauthordb
AUTHORS
BOOKS
INFORMATION_SCHEMA
Users
H2 2.2.224 (2023-09-17)
Run
Run Selected
Auto complete
Clear
SQL statement:
Important Commands
Displays this Help Page
Shows the Command History
Ctrl+Enter
Executes the current SQL statement
Shift+Enter
Executes the SQL statement defined by the text selection
Ctrl+Space
Auto complete
Disconnects from the database
Sample SQL Script
Delete the table if it exists
DROP TABLE IF EXISTS TEST;
Create a new table with ID and NAME columns
CREATE TABLE TEST(ID INT PRIMARY KEY, NAME VARCHAR(255));
Add a new row
INSERT INTO TEST VALUES(1, 'Hello');
Add another row
INSERT INTO TEST VALUES(2, 'World');
Query the table
SELECT * FROM TEST ORDER BY ID;
Change data in a row
UPDATE TEST SET NAME='Hi' WHERE ID=1;
Remove a row
DELETE FROM TEST WHERE ID=2;
Help
HELP ...
Adding Database Drivers
Additional database drivers can be registered by adding the Jar file location of the driver to the environment variables H2DRIVERS or CLASSPATH. Example (Windows): to add the database driver


Auto commit
Max rows: 1000
Auto complete Off
Auto select On
jdbc:h2:mem:bookauthordb
AUTHORS
BOOKS
INFORMATION_SCHEMA
Users
H2 2.2.224 (2023-09-17)
Run
Run Selected
Auto complete
Clear
SQL statement:
SELECT * FROM AUTHORS
SELECT * FROM AUTHORS;
ID NAME BIOGRAPHY NATIONALITY
1 J.K. Rowling Author of the Harry Potter series, one of the best-selling book series in history. British
2 Stephen King Master of horror fiction with over 60 novels published. American
3 Agatha Christie Best-selling mystery novelist known for detective Hercule Poirot. British
4 George Orwell Known for dystopian fiction and social criticism. British
5 Jane Austen Romantic fiction novelist from the Regency era. British
6 Ernest Hemingway Nobel Prize-winning author known for concise prose. American
7 F. Scott Fitzgerald Jazz Age novelist famous for The Great Gatsby. American
8 Gabriel Garcia Marquez Nobel Prize-winning magical realist author. Colombian
9 Haruki Murakami Contemporary fiction writer with surreal themes. Japanese
10 Toni Morrison Nobel Prize-winning novelist exploring African-American experience. American
(10 rows, 1 ms)
Edit



11. **Test Results** - Screenshot of `mvn test` output showing all tests passed

```
…/Downloads/book-author-app ❱ mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] --------------------< com.example:book-author-app >---------------------
[INFO] Building Book Author Management Application 1.0.0
[INFO]   from pom.xml
[INFO] --------------------------------[ war ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ book-author-app ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 2 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ book-author-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ book-author-app ---
[INFO] Copying 1 resource from src/test/resources to target/test-classes
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ book-author-app ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ book-author-app ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.bookauthor.exception.GlobalExceptionHandlerTest
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A Java agent has been loaded dynamically (/home/rehan/.m2/repository/net/bytebuddy/byte-buddy-agent/1.14.10/byte-buddy-agent-1.14.10.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.465 s -- in com.example.bookauthor.exception.GlobalExceptionHandlerTest
[INFO] Running com.example.bookauthor.repository.AuthorRepositoryTest
00:34:38.837 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration classes for test class [com.example.bookauthor.repository.AuthorRepositoryTest]: AuthorRepositoryTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
00:34:38.896 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found @SpringBootConfiguration com.example.bookauthor.BookAuthorApplication for test class com.example.bookauthor.repository.AuthorRepositoryTest

  .   ____          _            __ _ _
```

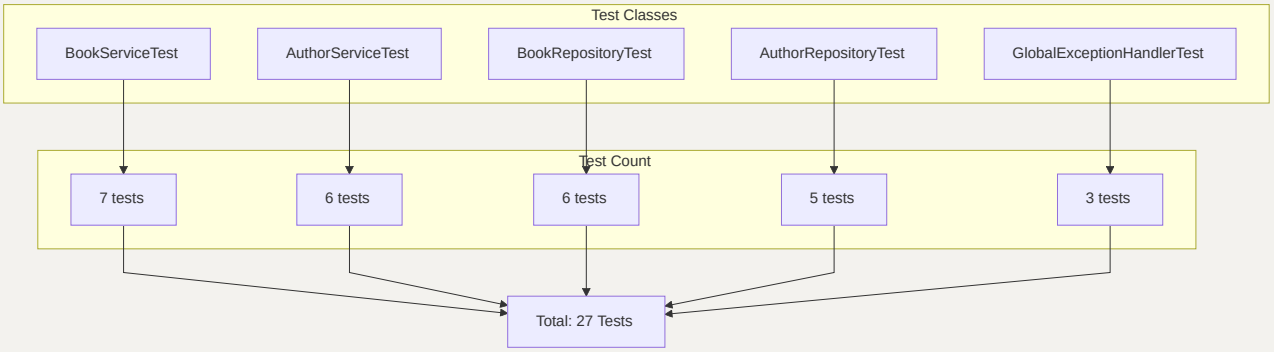
```
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.2.0)

2026-05-03T00:34:39.092+05:30  INFO 67489 --- [           main] c.e.b.repository.AuthorRepositoryTest    : Starting AuthorRepositoryTest using Java 26.0.1 with PID 67489 (starte
d by rehan in /home/rehan/Downloads/book-author-app)
2026-05-03T00:34:39.092+05:30  INFO 67489 --- [           main] c.e.b.repository.AuthorRepositoryTest    : No active profile set, falling back to 1 default profile: "default"
2026-05-03T00:34:39.245+05:30  INFO 67489 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-05-03T00:34:39.277+05:30  INFO 67489 --- [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 27 ms. Found 2 JPA reposit
ory interfaces.
2026-05-03T00:34:39.298+05:30  INFO 67489 --- [           main] beddedDataSourceBeanFactoryPostProcessor : Replacing 'dataSource' DataSource bean with embedded version
2026-05-03T00:34:39.365+05:30  INFO 67489 --- [           main] o.s.j.d.e.EmbeddedDatabaseFactory        : Starting embedded database: url='jdbc:h2:mem:137efb29-449a-435e-966c-b
4ea5f41eeee;DB_CLOSE_DELAY=-1;DB_CLOSE_ON_EXIT=false', username='sa'
2026-05-03T00:34:39.526+05:30  INFO 67489 --- [           main] o.hibernate.jpa.internal.util.LogHelper  : HHH000204: Processing PersistenceUnitInfo [name: default]
2026-05-03T00:34:39.549+05:30  INFO 67489 --- [           main] org.hibernate.Version                    : HHH000412: Hibernate ORM core version 6.3.1.Final
2026-05-03T00:34:39.565+05:30  INFO 67489 --- [           main] o.h.c.internal.RegionFactoryInitiator    : HHH000026: Second-level cache disabled
2026-05-03T00:34:39.605+05:30  INFO 67489 --- [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-05-03T00:34:39.946+05:30  INFO 67489 --- [           main] o.h.e.t.j.p.i.JtaPlatformInitiator       : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.p
latform' to enable JTA platform integration)
Hibernate: drop table if exists authors cascade
Hibernate: drop table if exists books cascade
Hibernate: create table authors (id bigint generated by default as identity, name varchar(100) not null unique, biography varchar(500), nationality varchar(255), primary key (id
))
Hibernate: create table books (publication_year integer, author_id bigint not null, id bigint generated by default as identity, title varchar(200) not null, description varchar(
1000), genre varchar(255), isbn varchar(255) not null unique, primary key (id))
Hibernate: alter table if exists books add constraint FKfjixh2vym2cvfj3ufxj91jem7 foreign key (author_id) references authors
2026-05-03T00:34:39.971+05:30  INFO 67489 --- [           main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-05-03T00:34:40.158+05:30  INFO 67489 --- [           main] o.s.d.j.r.query.QueryEnhancerFactory     : Hibernate is in classpath; If applicable, HQL parser will be used.
2026-05-03T00:34:40.396+05:30  INFO 67489 --- [           main] c.e.b.repository.AuthorRepositoryTest    : Started AuthorRepositoryTest in 1.473 seconds (process running for 2.3
83)
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: select a1_0.id,a1_0.biography,a1_0.name,a1_0.nationality from authors a1_0
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: select a1_0.id from authors a1_0 where a1_0.name=? fetch first ? rows only
Hibernate: select a1_0.id from authors a1_0 where a1_0.name=? fetch first ? rows only
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.776 s -- in com.example.bookauthor.repository.AuthorRepositoryTest
```

```
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: select a1_0.id from authors a1_0 where a1_0.name=? fetch first ? rows only
Hibernate: select a1_0.id from authors a1_0 where a1_0.name=? fetch first ? rows only
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.776 s -- in com.example.bookauthor.repository.AuthorRepositoryTest
[INFO] Running com.example.bookauthor.repository.BookRepositoryTest
2026-05-03T00:34:40.549+05:30  INFO 67489 --- [           main] t.c.s.AnnotationConfigContextLoaderUtils : Could not detect default configuration classes for test class [com.exa
mple.bookauthor.repository.BookRepositoryTest]: BookRepositoryTest does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
2026-05-03T00:34:40.556+05:30  INFO 67489 --- [           main] .b.t.c.SpringBootTestContextBootstrapper : Found @SpringBootConfiguration com.example.bookauthor.BookAuthorApplic
ation for test class com.example.bookauthor.repository.BookRepositoryTest
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: insert into books (author_id,description,genre,isbn,publication_year,title,id) values (?,?,?,?,?,?,default)
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: insert into books (author_id,description,genre,isbn,publication_year,title,id) values (?,?,?,?,?,?,default)
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: insert into books (author_id,description,genre,isbn,publication_year,title,id) values (?,?,?,?,?,?,default)
Hibernate: select b1_0.id from books b1_0 where b1_0.isbn=? fetch first ? rows only
Hibernate: select b1_0.id from books b1_0 where b1_0.isbn=? fetch first ? rows only
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: insert into books (author_id,description,genre,isbn,publication_year,title,id) values (?,?,?,?,?,?,default)
Hibernate: select b1_0.id,b1_0.author_id,b1_0.description,b1_0.genre,b1_0.isbn,b1_0.publication_year,b1_0.title from books b1_0 where b1_0.id=?
Hibernate: insert into authors (biography,name,nationality,id) values (?,?,?,default)
Hibernate: insert into books (author_id,description,genre,isbn,publication_year,title,id) values (?,?,?,?,?,?,default)
Hibernate: select b1_0.id,b1_0.title,b1_0.genre,b1_0.publication_year,a1_0.name,a1_0.nationality from books b1_0 join authors a1_0 on a1_0.id=b1_0.author_id
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.055 s -- in com.example.bookauthor.repository.BookRepositoryTest
[INFO] Running com.example.bookauthor.service.BookServiceTest
[INFO] Tests run: 8, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.084 s -- in com.example.bookauthor.service.BookServiceTest
[INFO] Running com.example.bookauthor.service.AuthorServiceTest
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.050 s -- in com.example.bookauthor.service.AuthorServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.383 s
[INFO] Finished at: 2026-05-03T00:34:40+05:30
[INFO] ------------------------------------------------------------------------
```

---

6. Testing & Validation

Test Coverage Summary



Test Results Table

LAYER	TEST FILE	TESTS	FRAMEWORK
Repository	`BookRepositoryTest.java`	6	JUnit 5 + @DataJpaTest
Repository	`AuthorRepositoryTest.java`	5	JUnit 5 + @DataJpaTest
Service	`BookServiceTest.java`	7	JUnit 5 + Mockito
Service	`AuthorServiceTest.java`	6	JUnit 5 + Mockito
Exception	`GlobalExceptionHandlerTest.java`	3	JUnit 5 + @WebMvcTest

Total: 27 unit tests

Sample Test Code (Repository Layer)

```
@DataJpaTest
public class BookRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private BookRepository bookRepository;

    @Test
    void testFindAllBooksWithAuthors() {
        Author author = new Author("John Doe", "American", "Bio");
        entityManager.persist(author);
```

```
        Book book = new Book("Test Book", "Fiction", 2023, "123",
"Desc", author);
        entityManager.persist(book);
        entityManager.flush();

        List<BookAuthorDTO> result =
bookRepository.findAllBooksWithAuthors();

        assertNotNull(result);
        assertFalse(result.isEmpty());
        assertEquals("Test Book", result.get(0).getBookTitle());
        assertEquals("John Doe", result.get(0).getAuthorName());
    }

    @Test
    void testExistsByIsbn() {
        assertTrue(bookRepository.existsByIsbn("999"));
        assertFalse(bookRepository.existsByIsbn("000"));
    }
}
```

Sample Test Code (Service Layer)

```
@ExtendWith(MockitoExtension.class)
public class BookServiceTest {

    @Mock
    private BookRepository bookRepository;

    @InjectMocks
    private BookServiceImpl bookService;

    @Test
    void testSaveBook_DuplicateIsbn() {
        Book testBook = new Book("Title", "Genre", 2023, "123-456",
"Desc", new Author());
        when(bookRepository.existsByIsbn("123-
456")).thenReturn(true);

        assertThrows(DataIntegrityViolationException.class, () -> {
```

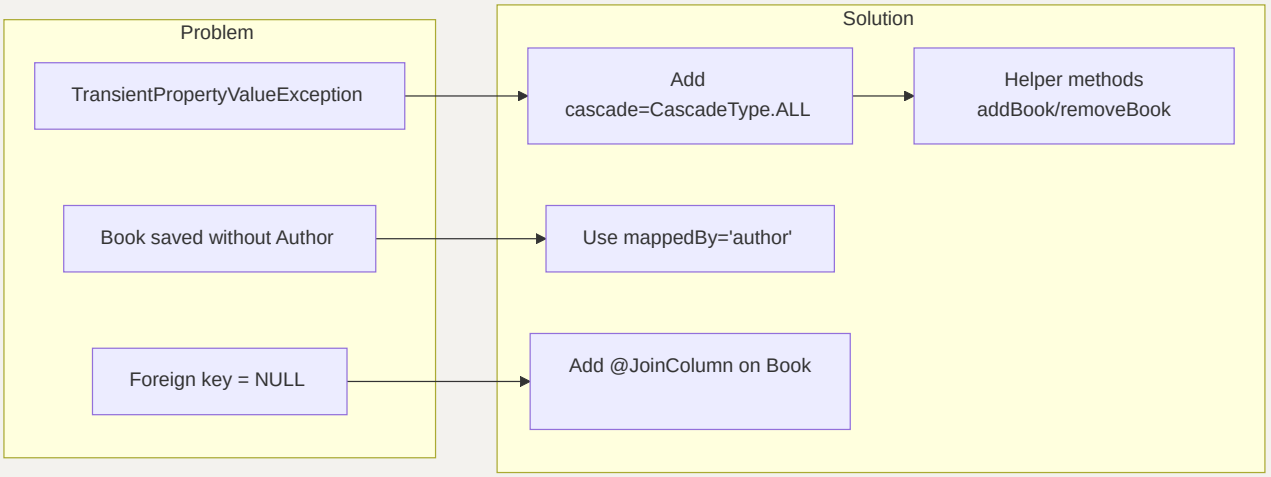
```
            bookService.saveBook(testBook);
        });
        verify(bookRepository, never()).save(any());
    }

    @Test
    void testUpdateBook() {

 when(bookRepository.findById(1L)).thenReturn(Optional.of(testBook)
);

 when(bookRepository.save(any(Book.class))).thenReturn(testBook);

        Book updatedDetails = new Book();
        updatedDetails.setTitle("Updated Title");
        // ... set other fields

        Book updated = bookService.updateBook(1L, updatedDetails);

        assertNotNull(updated);
        verify(bookRepository, times(1)).save(any(Book.class));
    }
}
```

Exception Handling Tests (`GlobalExceptionHandlerTest.java`)

```
@WebMvcTest(BookAuthorController.class)
public class GlobalExceptionHandlerTest {

    @Test
    void testHandleResourceNotFound() {
        // Test that ResourceNotFoundException returns error view
    }

    @Test
    void testHandleDataIntegrityViolation() {
        // Test that duplicate ISBN shows error message
    }
}
```

7. Challenges Faced & Solutions

Challenge 1: JPA Entity Relationship Configuration

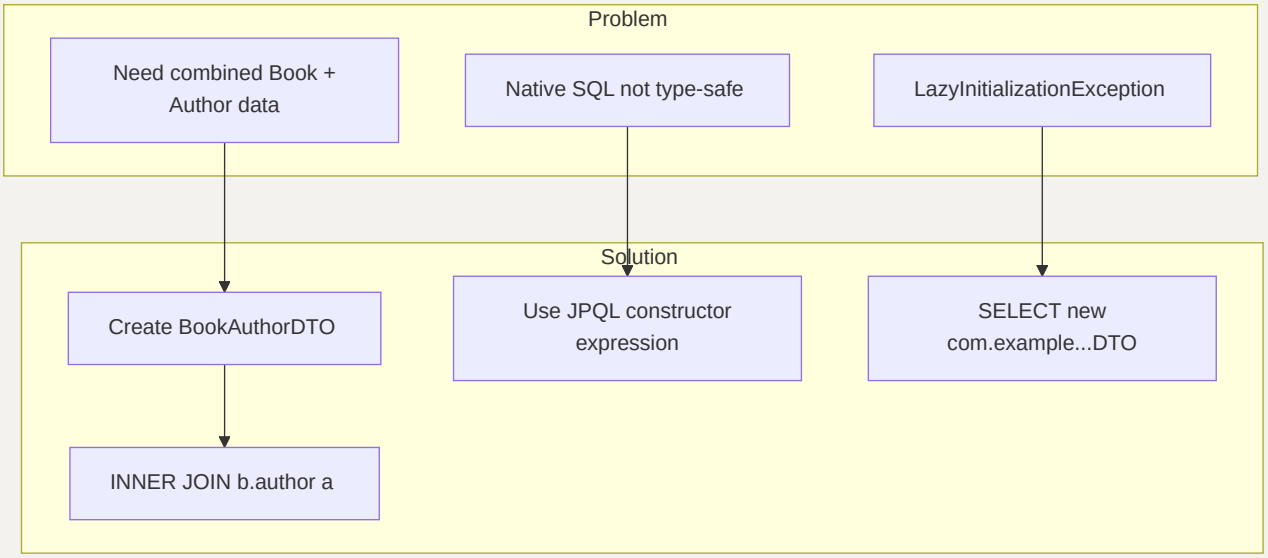


Problem: Initially struggled with configuring the bidirectional `@OneToMany` and `@ManyToOne` relationship correctly. The application threw `TransientPropertyValueException` when trying to save a Book without setting the Author first.

Solution:

- Added `cascade = CascadeType.ALL` and `orphanRemoval = true` on the `@OneToMany` side
- Used `mappedBy = "author"` to indicate Book is the owning side
- Added helper methods `addBook()` and `removeBook()` in Author entity for consistency
- Ensured `@JoinColumn(name = "author_id")` is on the Book entity (owning side)

Challenge 2: Custom JPQL Query for INNER JOIN



Problem: Needed to return combined data from both Book and Author tables using a custom query. Initially tried native SQL but needed a type-safe approach.

Solution:

- Created a DTO class (`BookAuthorDTO`) with a constructor matching the JPQL SELECT
- Used JPQL constructor expression: `SELECT new com.example.bookauthor.dto.BookAuthorDTO(...)`
- Used `INNER JOIN b.author a` to join Book with Author
- The DTO must have a constructor with matching parameter types

Challenge 3: Form Validation and Error Handling

Problem: When form validation failed (e.g., empty title), the form would reload but lose the author dropdown list.

Solution:

- Added `BindingResult` parameter to controller methods
- When `result.hasErrors()`, re-add the authors list to the model
- Used `@Valid @ModelAttribute("book") Book book` for automatic validation
- Added `@NotNull`, `@NotBlank`, `@Size` annotations on entity fields

Challenge 4: Handling Duplicate Data (ISBN/Name Uniqueness)

Problem: Users could add books with duplicate ISBN or authors with duplicate names, violating unique constraints.

Solution:

- Added `unique = true` constraint in `@Column` annotation
- Created custom query methods: `existsByIsbn()`, `existsByName()`
- In service layer, check for duplicates before saving/updating
- Throw `DataIntegrityViolationException` which is caught by `GlobalExceptionHandler`
- Display user-friendly error message on `error.jsp`

Challenge 5: JSP Page Not Found (404 Error)

Problem: After setting up JSP pages, the application returned 404 errors when accessing URLs.

Solution:

- Added `spring.mvc.view.prefix=/WEB-INF/jsp/` and `spring.mvc.view.suffix=.jsp` in `application.properties`
- Ensured JSP files are placed in `src/main/webapp/WEB-INF/jsp/` directory
- Added `tomcat-embed-jasper` dependency in `pom.xml` for JSP compilation
- Added JSTL dependency for `<c:forEach>` and other tags

Challenge 6: Data Population on Startup

Problem: Needed to populate the H2 database with sample data automatically on application startup.

Solution:

- Created `data.sql` file in `src/main/resources/`
- Spring Boot automatically executes `data.sql` on startup

- Used `spring.sql.init.mode=always` in `application.properties`
- Ensured author IDs in INSERT statements match the foreign key references

---

9. GitHub Repository

Repository URL: https://github.com/rehan-ahmed-cse/book-author-app

---

10. How to Run

Prerequisites

- Java 17+
- Maven 3.8+

Steps

1. **Clone the repository:**

```
git clone https://github.com/rehan-ahmed-cse/book-author-app.git
cd book-author-app
```

2. **Build the project:**

```
mvn clean install
```

3. **Run the application:**

```
mvn spring-boot:run
```

4. **Access the application:**
    - Open browser and navigate to `http://localhost:8080`
5. **Run tests:**

```
mvn test
```

6. **H2 Database Console:**
    - URL: `http://localhost:8080/h2-console`
    - JDBC URL: `jdbc:h2:mem:bookauthordb`
    - User: `sa`
    - Password: (leave empty)